

Visualization Library Documentation

INTRODUCTION:

Data Visualization is a process to present data in the form of graphs or visual data such as charts, graphs, plots and maps. In other words we can say that , data visualization is mathematically visually representing complex data data. This helps identify the trends and patterns followed in these data sets . This makes it easier to communicate between the viewer and the data set about what each of those data sets stand for.

In today's data-driven world, organizations generate large volumes of data every day. Analyzing such data in its raw form can be difficult and time-consuming. Data visualization helps simplify complex datasets and enables effective communication of information to both technical and non-technical audiences.

IDEAL PLATFORM FOR IMPLEMENTING DATA VISUALIZATION:

Python is considered one of the best programming languages for data visualization because it combines simplicity, flexibility, and a powerful collection of visualization libraries. Its easy-to-read syntax allows users to create visual representations of data with minimal code, making it suitable for both beginners and professionals.

Python offers a rich ecosystem of visualization libraries such as Matplotlib, Plotly, Seaborn, and Bokeh, each designed to address different visualization needs. Additionally, Python integrates seamlessly with data analysis libraries like NumPy and Pandas, enabling users to process, analyze, and visualize data within a single environment.

Due to its extensive community support, wide industry adoption, and ability to create both static and interactive visualizations, Python has become the preferred choice for data scientists, researchers, analysts, and organizations worldwide.

OBJECTIVE:

The primary objective of this project is to study and understand two widely used Python visualization libraries, **Matplotlib** and **Plotly**. Through this documentation, various graph types, features, and applications of these libraries are explored.

- **To study Matplotlib and Plotly** and understand their role in data visualization.
- **To create and analyze different types of visualizations** using both libraries.
- **To compare the features, capabilities, and use cases** of Matplotlib and Plotly.
- **To understand the importance of data visualization** in interpreting and communicating data effectively.
- **To gain practical knowledge** of implementing visualizations using Python.

Matplotlib:

~> Definition:

Matplotlib is an open-source Python library used for creating a wide variety of data visualizations. It enables users to represent data graphically through charts, plots, and graphs, making it easier to identify patterns, trends, and relationships within datasets. Due to its simplicity and flexibility, Matplotlib is widely used in data analysis, scientific computing, and academic research.

~> Features of Matplotlib:

Some of the key features of Matplotlib are:

- Supports a wide range of graph types such as line plots, bar charts, scatter plots, histograms, and pie charts.
- Provides extensive customization options for colors, labels, legends, and styles.
- Produces high-quality visualizations suitable for reports and presentations.
- Integrates seamlessly with NumPy, Pandas, and other Python libraries.
- Supports both static and interactive visualizations.
- Well-documented and widely supported by the Python community.

~> Installation of Matplotlib:

1. Matplotlib can be installed using Python's package manager, pip. Open the command prompt or terminal and execute the following command:

```
pip install matplotlib
```

2. To verify the installation, run the following Python code:

```
import matplotlib
print(matplotlib.__version__)
```

3. If the version number is displayed without errors, Matplotlib has been installed successfully.

~> Graph Types in Matplotlib:

Matplotlib provides several visualization techniques for representing data effectively. The following sections describe some of the most commonly used graph types along with their implementation and applications.

~> Line Plot

Description:

A line plot displays data points connected by straight lines. It is primarily used to show trends, changes, or patterns in data over a continuous interval or time period.

Use Case:

Tracking monthly sales, stock prices, temperature variations, or population growth over time.

~> Bar Chart

Description:

A bar chart represents categorical data using rectangular bars, where the length or height of each bar corresponds to the value it represents. It is useful for comparing different categories.

Use Case:

Comparing student marks, product sales, departmental budgets, or survey results.

~> Scatter Plot

Description:

A scatter plot displays individual data points on a coordinate plane to illustrate the relationship between two variables. It helps identify correlations, clusters, and outliers.

Use Case:

Analyzing the relationship between height and weight, study hours and exam scores, or advertising expenditure and sales.

~> Histogram

Description:

A histogram represents the frequency distribution of numerical data by grouping values into intervals called bins. It helps visualize the distribution and spread of data.

Use Case:

Analyzing exam score distributions, customer age groups, income distributions, or measurement data.

~> Pie Chart**Description:**

A pie chart represents data as slices of a circle, where each slice corresponds to a proportion of the whole. It is useful for displaying percentage distributions.

Use Case:

Showing market share, budget allocation, population distribution, or survey responses.

NOTE: Apart from the graphs matplotlib is also used to display the following:

- Original vs Processed images
 - Multiple versions of an image
 - Histograms
 - Color space comparisons
-

>> SOME CODE SNIPPETS REPRESENTING USE OF MATPLOTLIB:**1. Basic line plot:**

code-import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]

y = [10, 20, 15, 30, 25]

plt.plot(x, y)

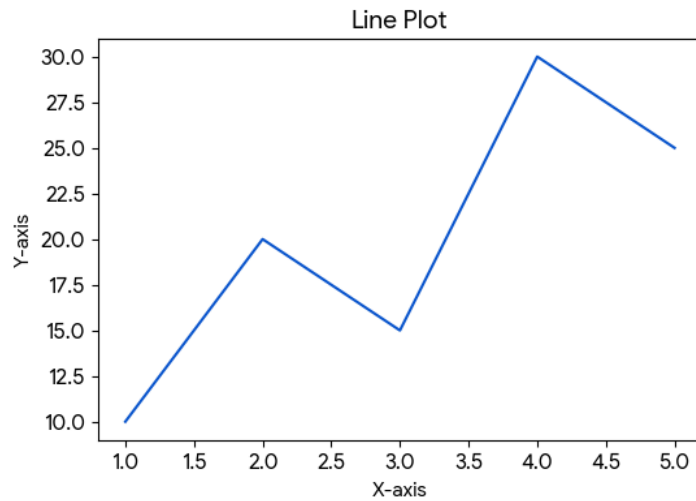
plt.title("Line Plot")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.show()

output-



2. Bar chart:

code-import matplotlib.pyplot as plt

subjects = ["Math", "Java", "OS"]

marks = [85, 90, 80]

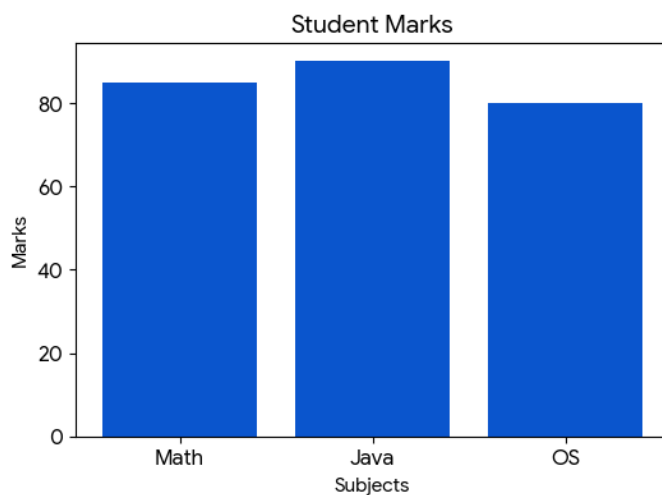
plt.bar(subjects, marks)

plt.title("Student Marks")

plt.xlabel("Subjects")

plt.ylabel("Marks")

plt.show()



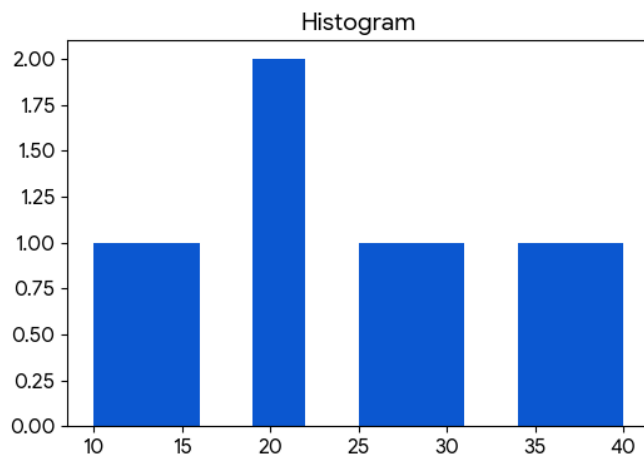
output-

3. Histogram:

```
code-import matplotlib.pyplot as plt
```

```
data = [10, 15, 20, 20, 25, 30, 35, 40]
```

```
plt.hist(data)  
plt.title("Histogram")  
plt.show()
```



Output-

4. Displaying an image:

```
code-import matplotlib.pyplot as plt
```

```
import matplotlib.image as mpimg
```

```
img = mpimg.imread("/Users/anikasingh/Downloads/images-2.jpeg")
```

```
plt.imshow(img)  
plt.axis("off")  
plt.show()
```



Output-

5. Detect and visualize edges using Canny edge detection:

```
code-import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("/Users/anikasingh/Downloads/images-2.jpeg", 0)

# Apply Canny Edge Detection
edges = cv2.Canny(img, 100, 200)

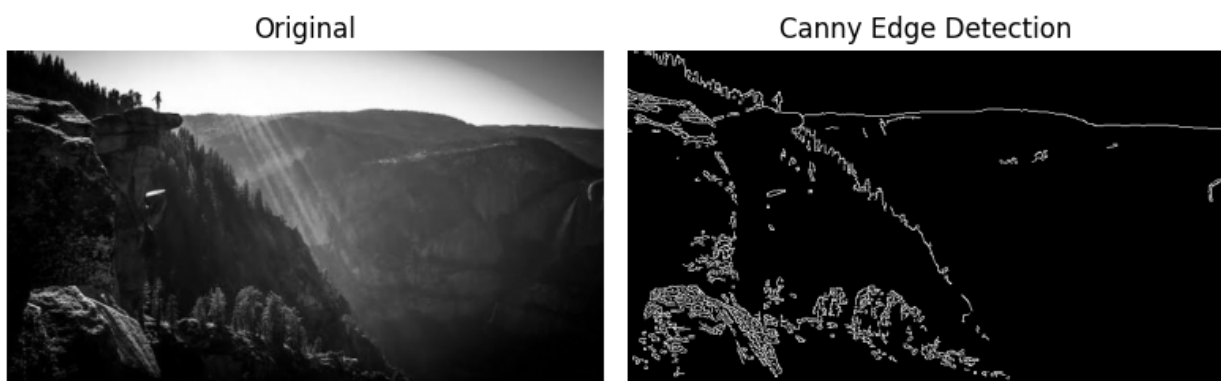
# Display Original and Edge Detected Images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis("off")

plt.show()
```

Output-



6. Implementation of color space conversion:

```
Code- import cv2
import matplotlib.pyplot as plt
```

```

# Read image
img = cv2.imread("image.jpg")

# Convert color spaces
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display images
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(rgb)
plt.title("RGB")
plt.axis("off")

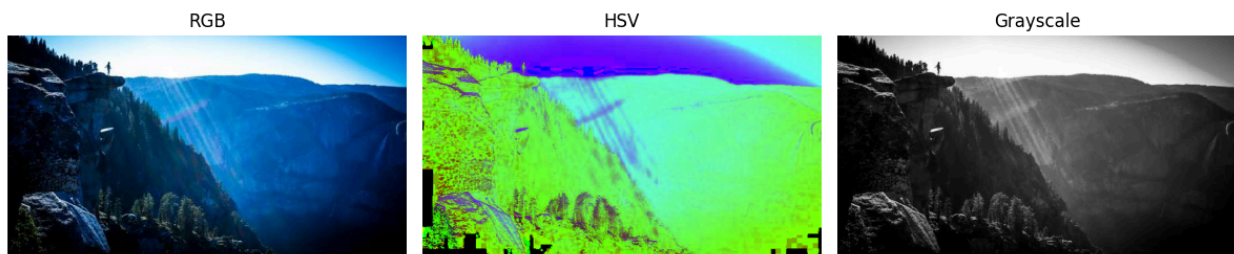
plt.subplot(1,3,2)
plt.imshow(hsv)
plt.title("HSV")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale")
plt.axis("off")

plt.show()

```

Output-



7. Implement template matching within an image:

Code- import cv2

import matplotlib.pyplot as plt


```

# Read image
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Select a template (sample region)
template = gray[60:140, 120:220]
h, w = template.shape

# Perform template matching
result = cv2.matchTemplate(gray, template, cv2.TM_CCOEFF_NORMED)

# Find best match location
_, _, _, max_loc = cv2.minMaxLoc(result)

top_left = max_loc
bottom_right = (top_left[0] + w, top_left[1] + h)

# Draw bounding box around matched region
cv2.rectangle(img, top_left, bottom_right, (0, 255, 0), 3)

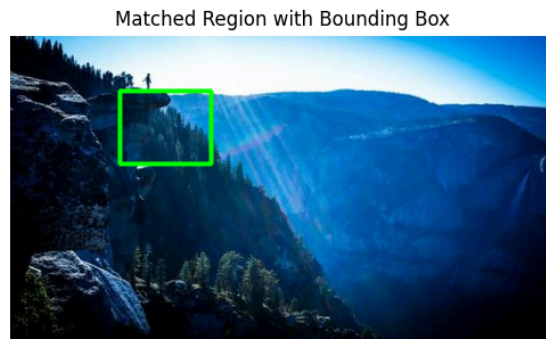
# Display results
plt.subplot(1,2,1)
plt.imshow(template, cmap='gray')
plt.title("Template")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Matched Region")
plt.axis("off")

plt.show()

```

Output-



8. Apply rotation and scaling to an image:

Code- from PIL import Image

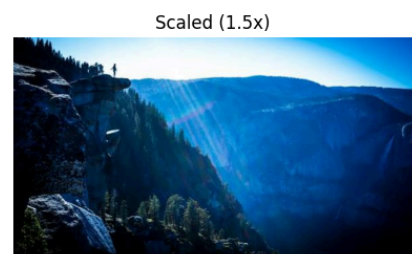
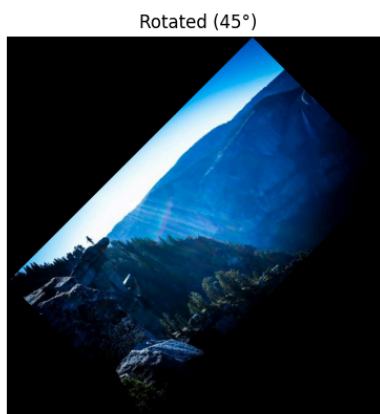
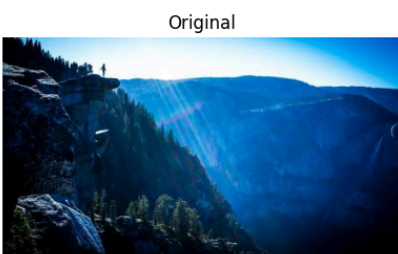
```
img = Image.open("images-2.jpeg")
```

```
# Rotate image by 45 degrees
rotated = img.rotate(45, expand=True)
```

```
# Scale image by 1.5 times
scaled = img.resize((int(img.width*1.5), int(img.height*1.5)))
```

```
rotated.save("rotated.jpg")
scaled.save("scaled.jpg")
```

Output-



Plotly:

~> Definition:

Plotly is an open-source data visualization library in Python that enables users to create interactive, high-quality graphs and charts for data analysis and presentation. It supports a wide variety of visualizations, including line charts, bar charts, scatter plots, pie charts, histograms, heatmaps, and 3D plots. Plotly's interactive features, such as zooming, panning, and hover tooltips, make it easier to explore and understand data. The library integrates seamlessly with popular Python tools like Pandas and NumPy and is widely used in data science, business intelligence, and dashboard development for creating dynamic and visually appealing visualizations.

~> Features of Plotly:

- **Interactive Visualizations** – Supports zooming, panning, hovering, and selection of data points.
- **Wide Range of Charts** – Creates line charts, bar charts, scatter plots, pie charts, histograms, heatmaps, box plots, and 3D visualizations.
- **High-Quality Graphics** – Produces publication-ready and professional-looking visualizations.
- **Easy Integration** – Works seamlessly with Pandas, NumPy, and Jupyter Notebooks.
- **Web-Based Rendering** – Generates charts that can be viewed and shared through web browsers.
- **Dashboard Support** – Integrates with Dash to build interactive data dashboards and web applications.
- **Customizable Plots** – Allows extensive customization of colors, layouts, labels, themes, and annotations.
- **Cross-Platform Compatibility** – Supports Python, R, JavaScript, and other programming languages.
- **Export Options** – Enables exporting charts as HTML, PNG, JPEG, SVG, and PDF files.
- **3D and Geographic Visualizations** – Supports 3D plotting and map-based visualizations for advanced data analysis.

~> Installation of Plotly:

~>Installation of Plotly (Step-by-Step):

Step 1: Open **Command Prompt (Windows)** or **Terminal (Mac/Linux)**.

Step 2: Check whether Python is installed by running:

```
python --version
```

Step 3: Install Plotly using the pip package manager:

```
pip install plotly
```

Step 4: Wait for the installation process to complete successfully.

Step 5: Verify the installation by opening Python and typing:

```
import plotly
print(plotly.__version__)
```

Step 6: If no errors occur and the version number is displayed, Plotly has been installed successfully.

Step 7 (Optional): If you are using Jupyter Notebook, install Plotly directly from a notebook cell:

```
!pip install plotly
```

Step 8: Start creating interactive visualizations using Plotly in your Python projects.

~>Main components of Plotly:

1. Plotly Express (px)

- A high-level interface for creating visualizations with minimal code.
- Ideal for quick and simple data exploration.

Example:

```
import plotly.express as px
```

```
fig = px.scatter(x=[1, 2, 3, 4], y=[10, 20, 15, 25],
                 title="Scatter Plot")
fig.show()
```

2. Plotly Graph Objects (go)

- A low-level interface that provides greater control and customization.
- Suitable for creating complex and detailed visualizations.

Example:

```
import plotly.graph_objects as go
```

```
fig = go.Figure(  
    data=go.Bar(x=['A', 'B', 'C'], y=[10, 20, 15])  
)  
fig.show()
```

~>Common Plotly Chart Types:

1. Box Plot

A **Box Plot** is used to visualize the distribution of numerical data through quartiles. It helps identify the median, spread of data, and potential outliers.

Use Cases: Statistical analysis, detecting outliers, comparing distributions.

2. Heatmap

A **Heatmap** represents data values using different colors. It is useful for showing patterns, correlations, and variations in large datasets.

Use Cases: Correlation matrices, performance analysis, data density visualization.

3. Bubble Chart

A **Bubble Chart** is an extension of a scatter plot where the size of each bubble represents an additional variable.

Use Cases: Comparing three variables simultaneously, business and economic analysis.

4. Area Chart

An **Area Chart** displays quantitative data over time with the area below the line filled with color. It highlights trends and cumulative values.

Use Cases: Sales trends, revenue growth, population changes.

5. 3D Surface Plot

A **3D Surface Plot** visualizes three-dimensional data where the x, y, and z axes represent different variables.

Use Cases: Scientific simulations, engineering analysis, mathematical modeling.

6. Geographic Maps

Geographic Maps display data on maps using locations, markers, and color scales for geographical analysis.

Use Cases: Population studies, sales by region, weather and location-based data.

>> SOME CODE SNIPPETS REPRESENTING USE OF PLOTLY:

1. Box Plot – Student Performance by Department:

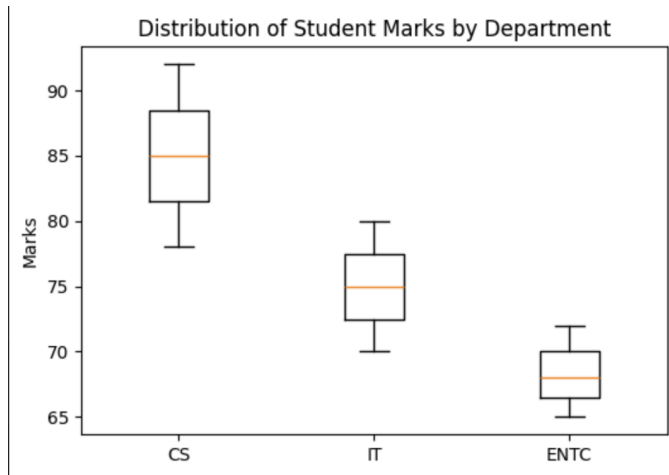
Code- import plotly.express as px
import pandas as pd

```
df = pd.DataFrame({  
    'Department': ['CS','CS','CS','IT','IT','IT','ENTC','ENTC','ENTC'],  
    'Marks': [85, 78, 92, 70, 75, 80, 65, 72, 68]  
})
```

```
fig = px.box(  
    df,  
    x='Department',  
    y='Marks',  
    color='Department',  
    title='Distribution of Student Marks by Department'  
)
```

fig.show()

Output-



2. Heatmap – Monthly Sales Analysis:

Code- import plotly.graph_objects as go

months = ['Jan','Feb','Mar','Apr']

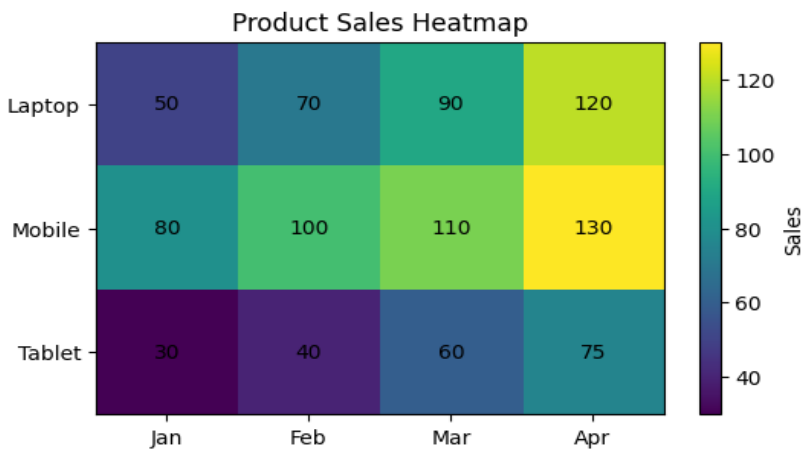
products = ['Laptop','Mobile','Tablet']

```
sales = [
    [50, 70, 90, 120],
    [80, 100, 110, 130],
    [30, 40, 60, 75]
]
```

```
fig = go.Figure(
    data=go.Heatmap(
        z=sales,
        x=months,
        y=products
    )
)
```

```
fig.update_layout(title='Product Sales Heatmap')
fig.show()
```

Output-



3. Bubble Chart- Company employee analysis:

Code- import plotly.express as px
import pandas as pd

```
df = pd.DataFrame({
    'Experience':[1,2,3,5,7,10],
    'Salary':[25000,30000,40000,55000,70000,100000],
    'Projects':[2,3,4,6,8,10]
})
```

```
fig = px.scatter(
    df,
    x='Experience',
    y='Salary',
    size='Projects',
    color='Projects',
    hover_name='Projects',
    title='Employee Experience vs Salary'
)
fig.show()
```

Output-



4. Dual Y-Axis Chart:

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go
```

```
fig = make_subplots(specs=[[{"secondary_y": True}]])
```

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales = [150, 180, 170, 220]
profit = [20, 25, 22, 35]
```

```
fig.add_trace(
    go.Bar(x=months, y=sales, name="Sales"),
    secondary_y=False
)
```

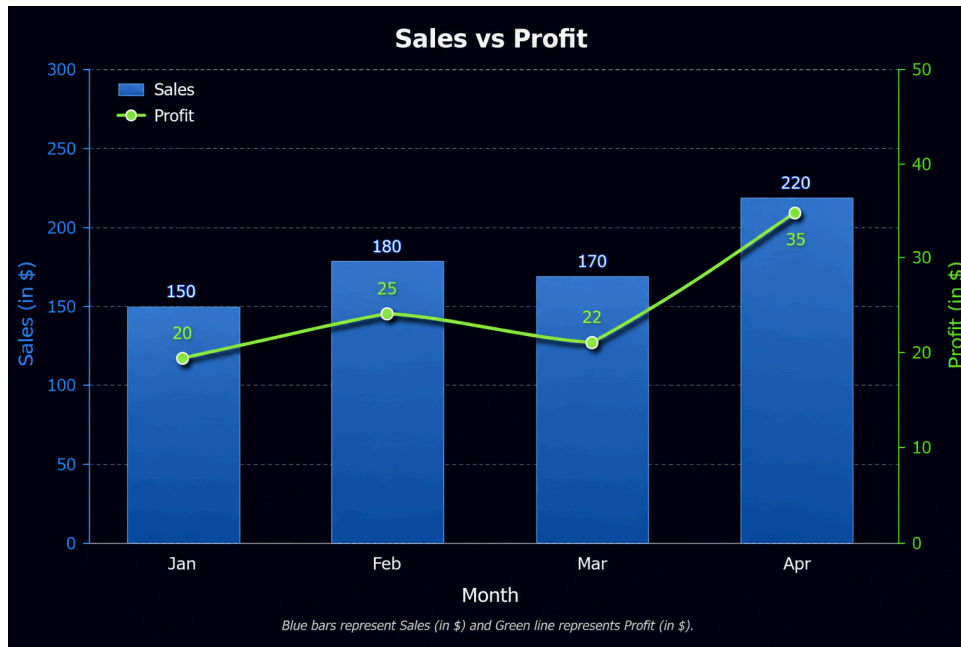
```
fig.add_trace(
    go.Scatter(
        x=months,
        y=profit,
        mode="lines+markers",
        name="Profit"
    ),
    secondary_y=True
)
```

```
fig.update_layout(title="Sales vs Profit")
```

```
fig.update_yaxes(title_text="Sales", secondary_y=False)
fig.update_yaxes(title_text="Profit", secondary_y=True)
```

```
fig.show()
```

Output:



COMPARISON BETWEEN MATPLOTLIB AND PLOTLY:

Feature	Matplotlib	Plotly
Primary Purpose	Static, publication-quality visualizations	Interactive, web-based visualizations and dashboards
Interactivity	Limited (mostly static unless extended)	Built-in zoom, pan, hover, selection, and animations
Ease of Learning	Moderate; requires more customization	Easier for beginners with high-level APIs like Plotly Express
Customization	Extremely flexible but verbose	Highly customizable with fewer lines of code

Visual Appeal	Clean and professional but basic by default	Modern, attractive, and presentation-ready by default
Performance	Excellent for small to medium datasets	Good for moderate datasets; may slow down with very large datasets
3D Visualization	Supported but relatively basic	Rich interactive 3D visualizations with rotation and zoom
Dashboard Integration	Requires external libraries	Integrates seamlessly with Dash for interactive dashboards
Animation Support	Available but requires more coding	Built-in animation support using simple parameters
Hover Information	Not available by default	Rich tooltips and hover information are built in
Export Options	PNG, PDF, SVG, EPS, etc.	HTML (interactive), PNG, PDF, SVG
Best Use Cases	Research papers, scientific publications, reports	Business intelligence, dashboards, exploratory data analysis, presentations
Web Integration	Limited	Excellent support for web applications
Community & Ecosystem	Very large and mature	Growing rapidly with strong documentation

CONCLUSION:

In this study, both **Matplotlib** and **Plotly** proved to be valuable visualization libraries, each serving different purposes in a data science workflow. **Matplotlib** is best suited for creating static, publication-quality graphs with extensive customization, while **Plotly** excels in producing interactive, visually appealing visualizations that enhance data exploration and stakeholder communication. The choice between the two depends on the project requirements—Matplotlib for research and reporting, and Plotly for exploratory data analysis, dashboards, and interactive presentations. A data scientist can leverage the strengths of both libraries to create effective and insightful visualizations.

REFERENCES:

1. Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, **9**(3), 90–95.
2. Plotly Technologies Inc. (2024). *Plotly Python Graphing Library Documentation*. Available at: <https://plotly.com/python/>
3. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array Programming with NumPy*. Nature, **585**, 357–362.
4. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
5. VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.